生成AIのコーディング技法

川原 純

京都大学

2026 年 10 月 9 日

本講演のスライドは

https://github.com/junkawahara/q にあります

本講演の概要

- コーディングエージェントで
  自分用のソフトウェアを作ってみよう

- 既に Codex や Claude Code を
  使いこなしている人には物足りない内容かも

本講演のスライドは

`https://github.com/junkawahara/q` にあります

生成 AI とは

- 文章・画像・プログラムなどを生成する AI
  - 大規模言語モデル（LLM）が中核
- 代表例
  - ChatGPT（OpenAI）
  - Gemini（Google）
  - Claude（Anthropic）
- チャット形式で質問・依頼
  - AI が回答を生成

コーディングエージェントとは

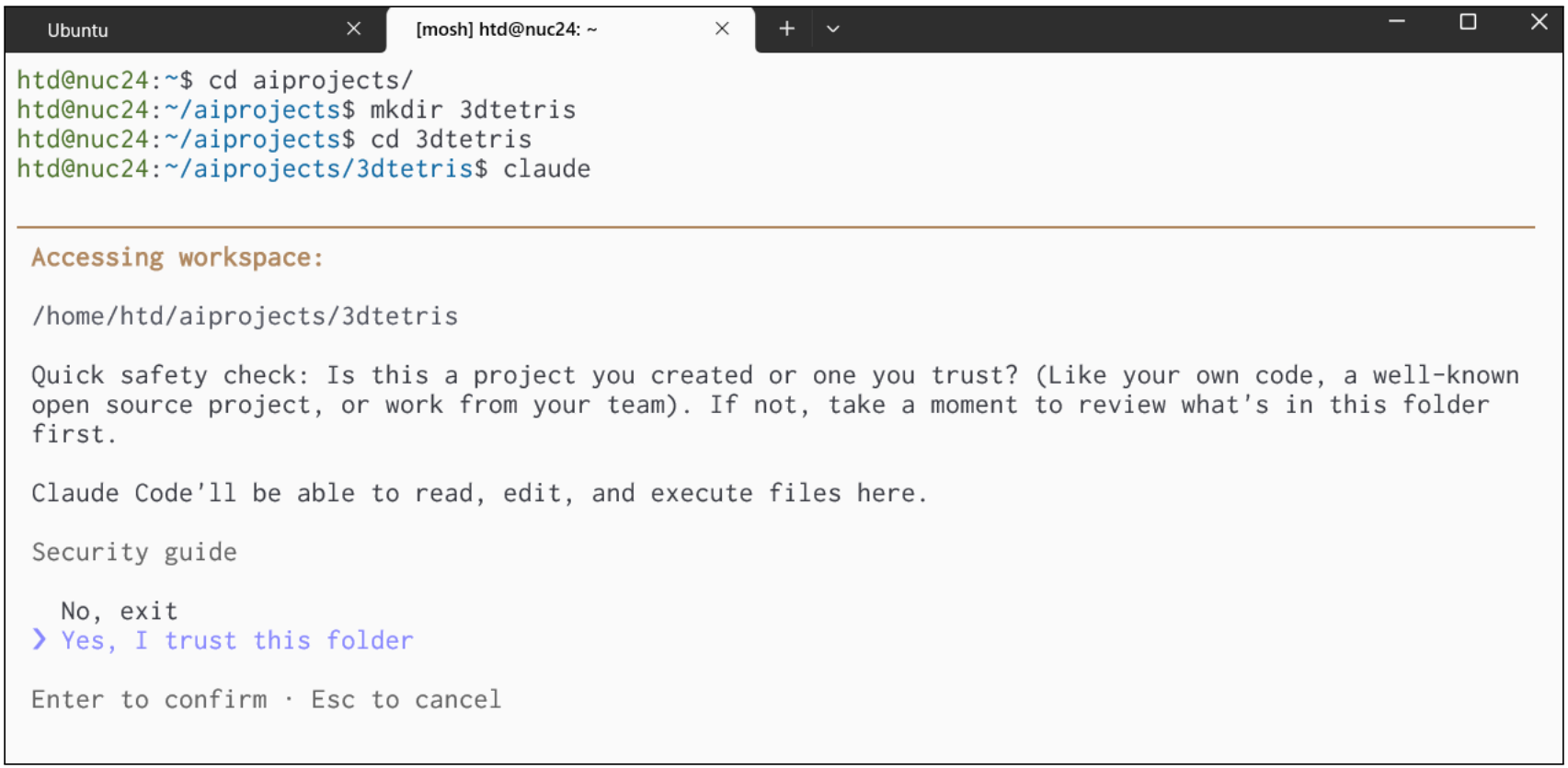
- 生成 AI が自分の PC 上で自律的に作業
  - ファイルの読み書き
  - コマンドの実行
- 代表例
  - Codex（OpenAI）
  - Claude Code（Anthropic）
- ターミナル上で AI と対話しながら利用

コーディングエージェントにできること

- 自らプログラミングを行う
  - コードを書く → 実行・テスト → 修正
- ソフトウェアを丸ごと開発できる
  - バイブコーディングや
    エージェンティックコーディングと呼ばれる

例: 3D テトリスを作らせる

- ディレクトリを作成し、claude を起動
- 3dtetris 以下の読み書き権限を与える



Claude Code を起動



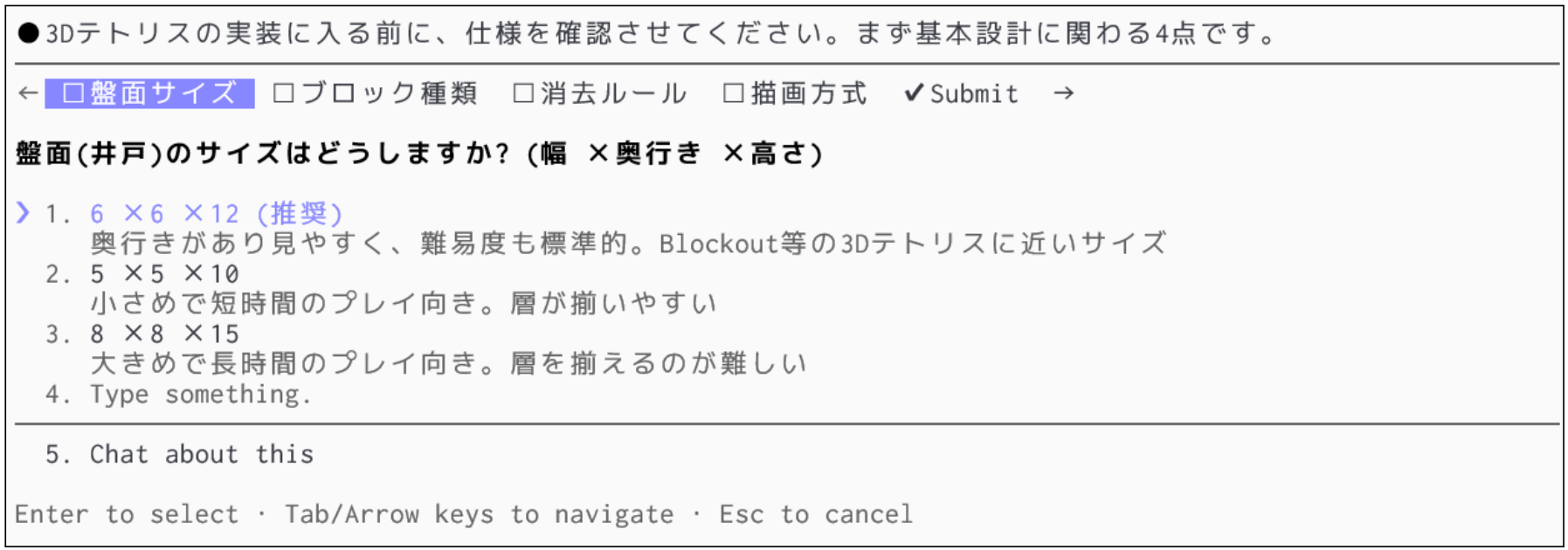
- プロンプトを入力（ChatGPT とほぼ同じ）
- 「徹底的に私に聞いて」が重要
  - 書かないと AI が勝手に仕様を決める

Claude Code を起動



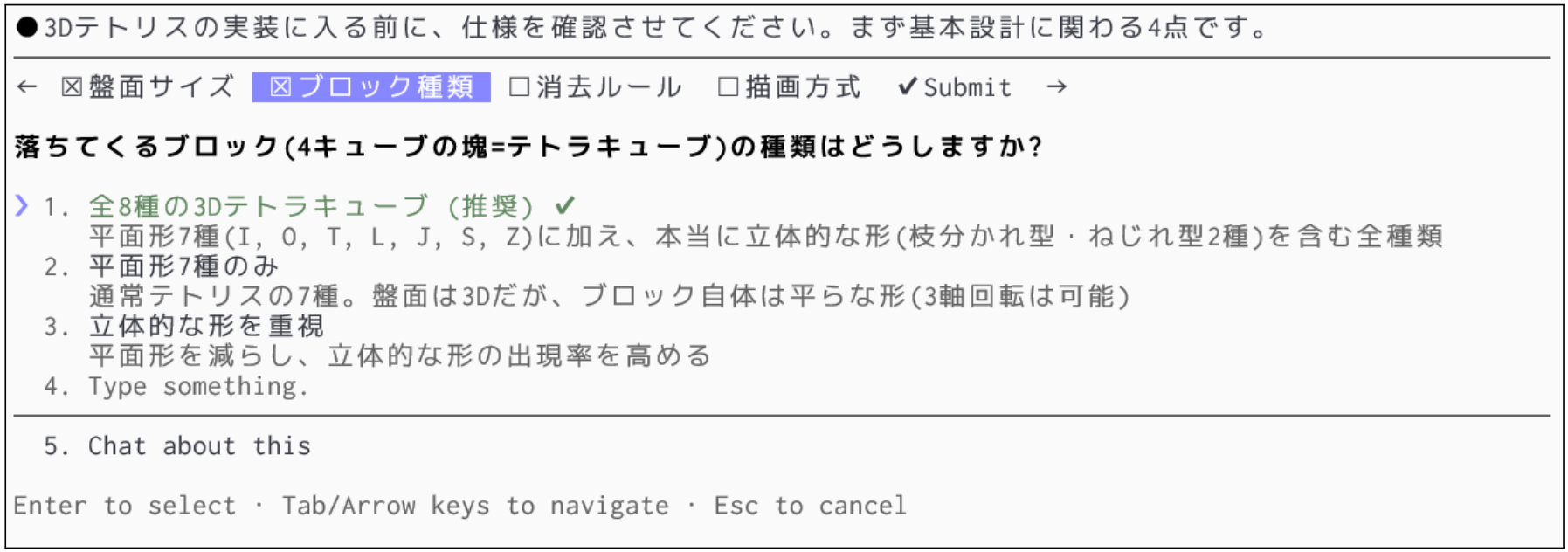
- プロンプトを入力（ChatGPT とほぼ同じ）
- 「徹底的に私に聞いて」が重要
  - 書かないと AI が勝手に仕様を決める

AIから仕様について質問される (1/4)



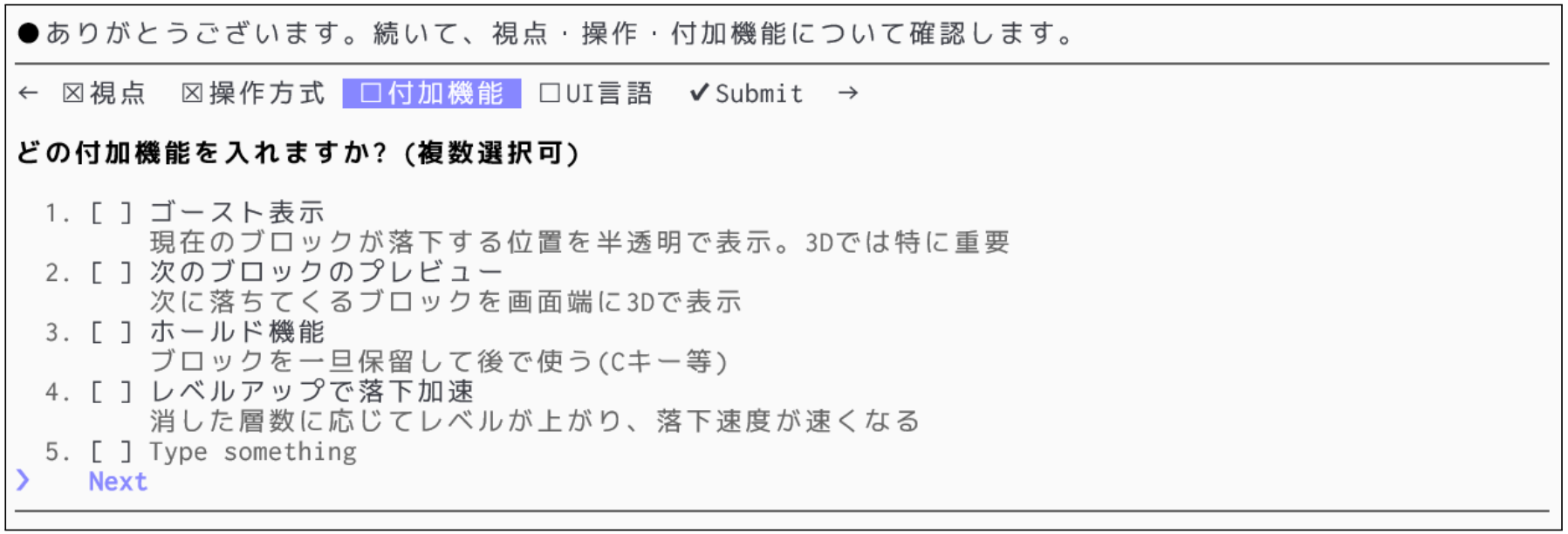
- 仕様について次々と質問される
  - 答えるだけでよい
- 分からない点はこちらから質問すればよい

AIから仕様について質問される (2/4)



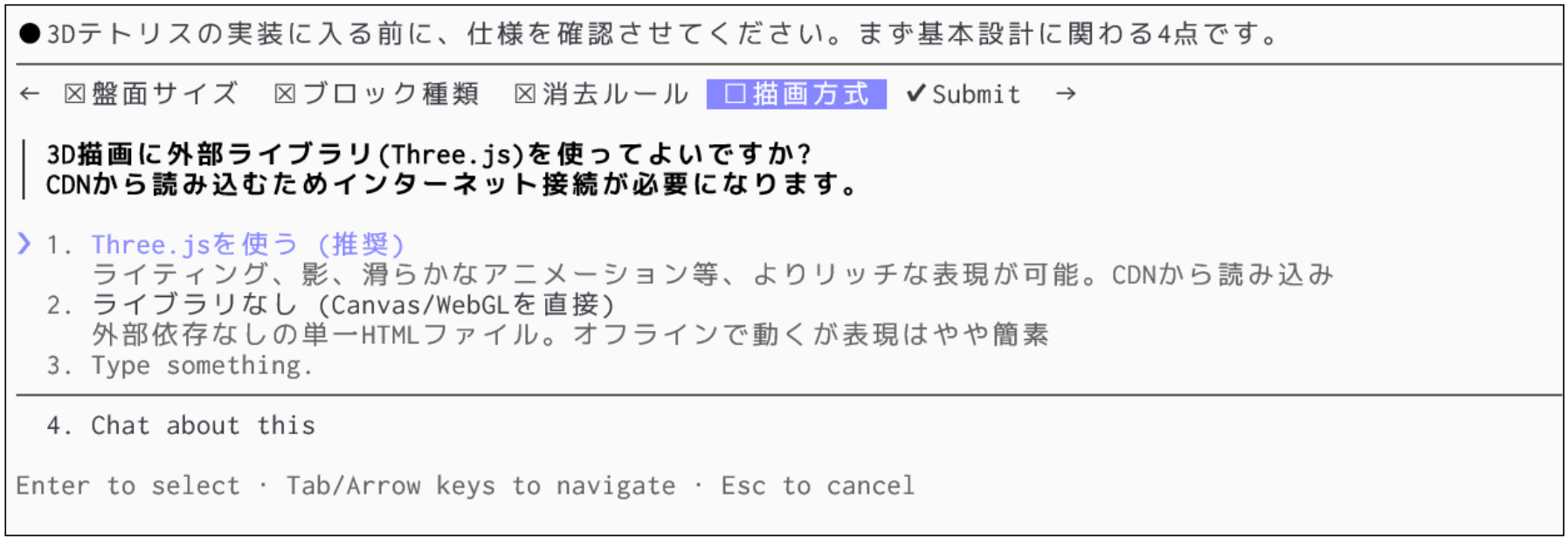
- 仕様について次々と質問される
  - 答えるだけでよい
- 分からない点はこちらから質問すればよい

AIから仕様について質問される (3/4)



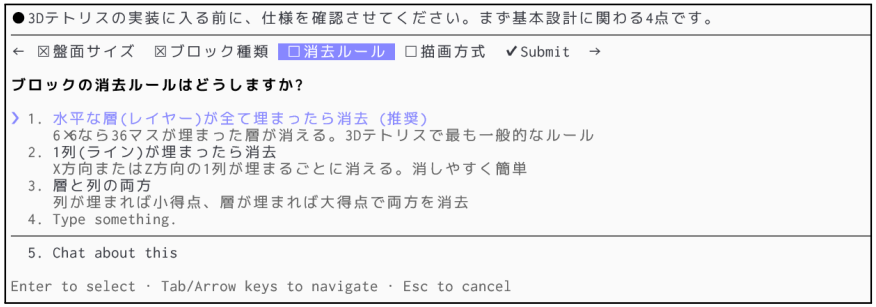
- 仕様について次々と質問される
  - 答えるだけでよい
- 分からない点はこちらから質問すればよい

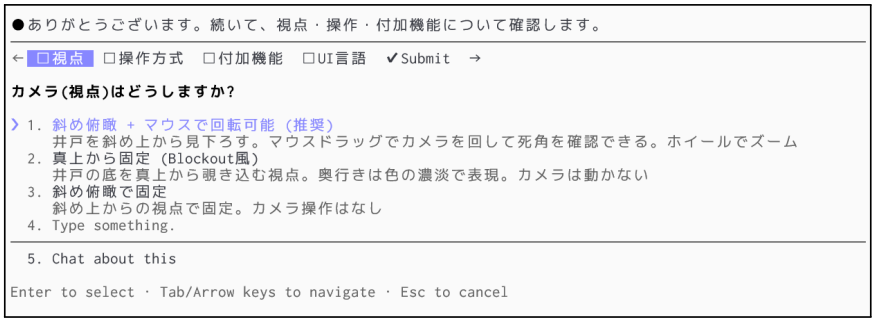
AIから仕様について質問される (4/4)

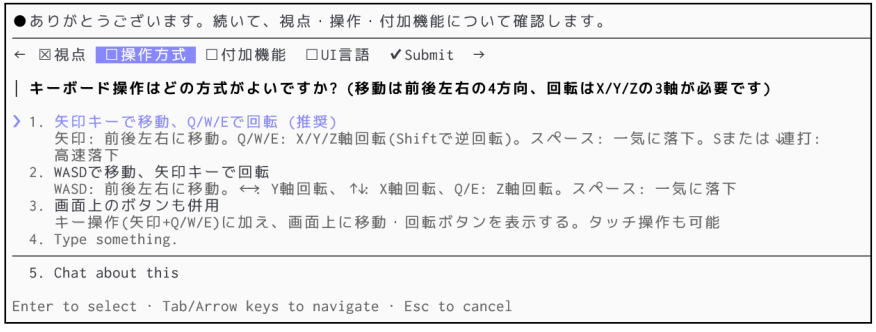


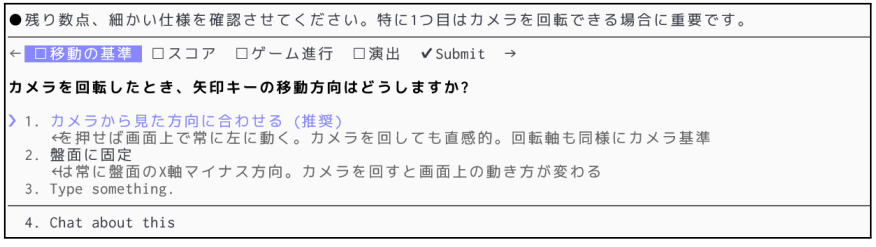
- 仕様について次々と質問される
  - 答えるだけでよい
- 分からない点はこちらから質問すればよい

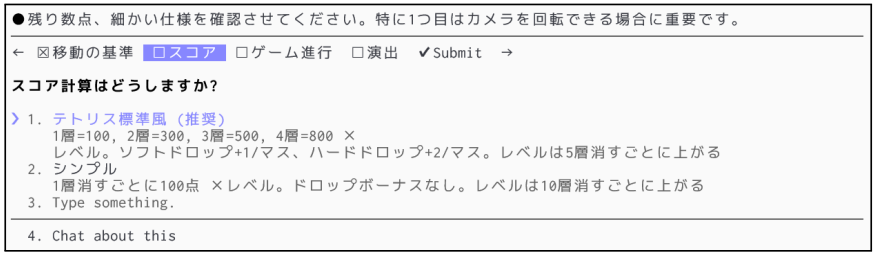
AIからの質問の例

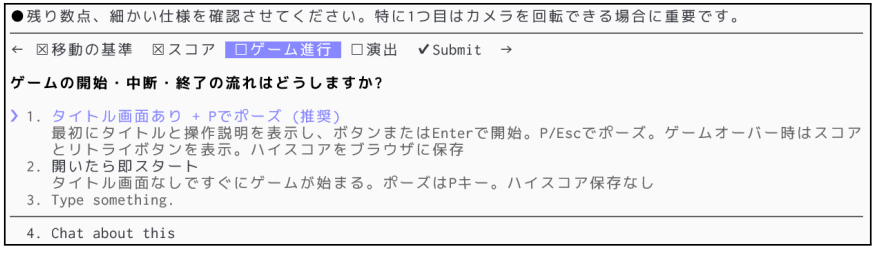








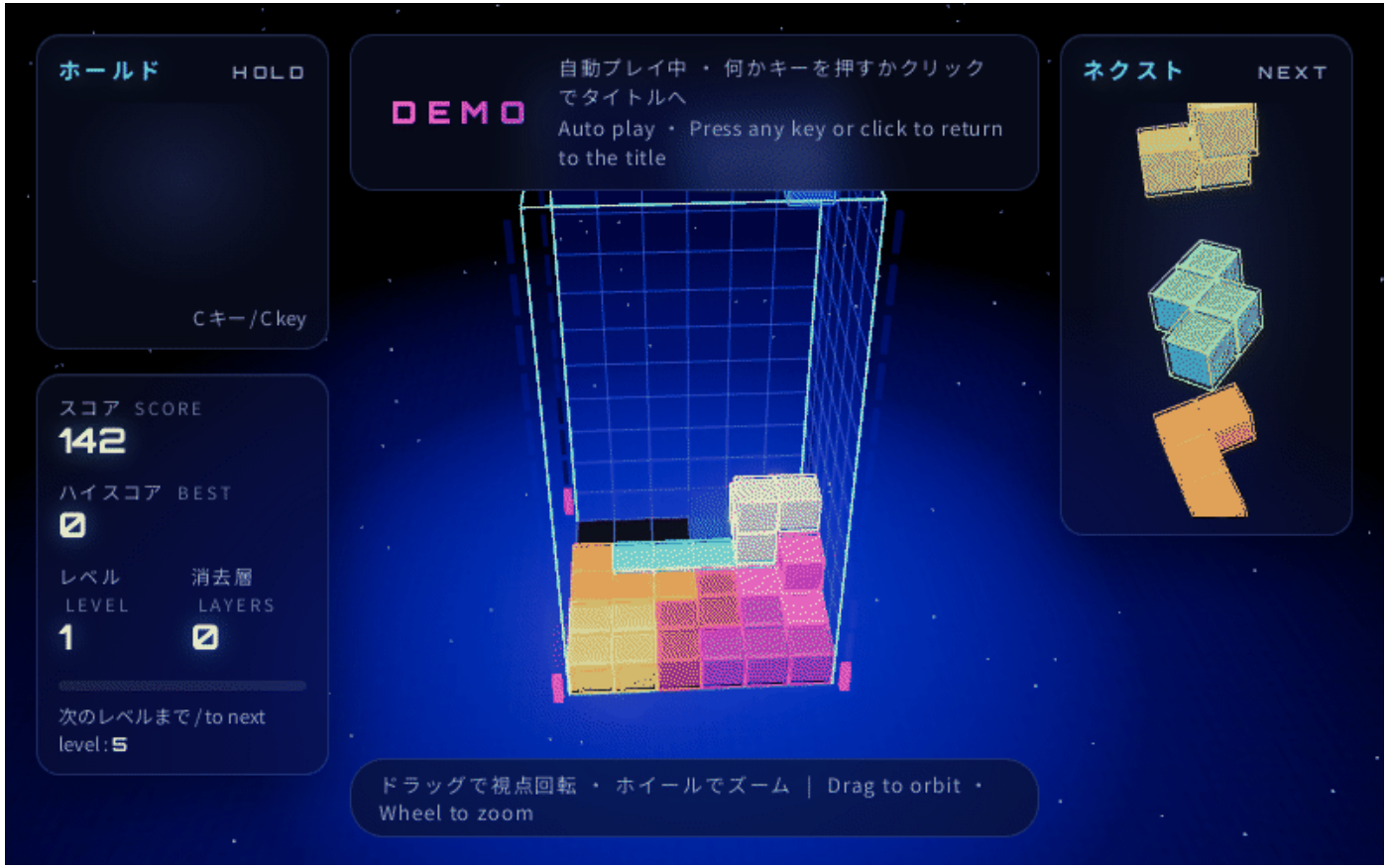




- 自分が想定していなかった着眼点も

完成: 3D テトリス

- 質問に答えたら完成



- 6×6×12 の井戸に立体ブロックを積む
- AI が自動でプレイするデモモード付き

書かせたコードはレビューさせる

- 書かせたコードはレビューさせることが大事
- 私が使っているプロンプト例

> リポジトリ内のコードやテスト、ドキュメントを徹底的に敵対的レビューして、見つかったバグは細かいものを含めて、すべて review_yyyyMMdd_X.md にリストアップしてください。yyyyMMdd は現在の年月日、X は連番です。ただし、既存の KNOWN_ISSUES.md や review*.md（ファイルがもしあれば）にある項目は無視してください。各項目ごとに、連番、チェックボックス [ ]、重要性（重大・高・中・低）をつけてください。書式例は "1. [ ] 【低】" です。重要性でソートしてください（重大を先に並べて、次は高、その次は中）。

レビュー結果: 23 件のバグを発見

- 3D テトリスのレビューで 23 件のバグを発見
- レビューログの抜粋

レビュー結果 2026-10-09 (#1)

対象: `index.html`, `js/logic.js`, `style.css`, `README.md`, `demo.png`
(リポジトリに `KNOWN_ISSUES.md` および既存の `review*.md` は無し。テストコードも存在しない。)

検証: `js/logic.js` は Node で実際に動かして確認済み (回転・ピース同値性・デモの回転計画)。`index.html` はコードを読んで確認したもの。

---

1. [ ]【高】デモをライン消去アニメーション中に終了すると、消える途中のキューブが画面に残り続ける
   - 場所: `index.html` `startClear()`(612–628 行) と `clearEverything()`(664–672 行)
   - `startClear` は消去対象のメッシュをすぐ `locked` から外しますが、`lockedGroup` から外すのは 0.2 秒後の `after()` の中です。この 0.2 秒の間にデモを終了すると (キー入力やクリック。`endDemo` → `returnToTitle` → `clearEverything`)、`tweens.length = 0` でそのコールバックが消されます。さらに `clearEverything` は `locked` に残っているメッシュしか消しません。

作例1: アドバイザー面談予約システム

軽めのものから本格的な研究ソフトウェアへ

- 学生が面談を予約する Web ページ
- さくらのレンタルサーバーで動作
  - データ保持（SQLite）も AI に相談
- コードの中身は見ていない
  - JavaScript は分からない



作例 2: 学生実験の出席管理システム

- 学生証をカードリーダーで読み取り出席管理
- 学生番号の読み取り方法を AI に聞きながら作成
  - 意外と手こずった

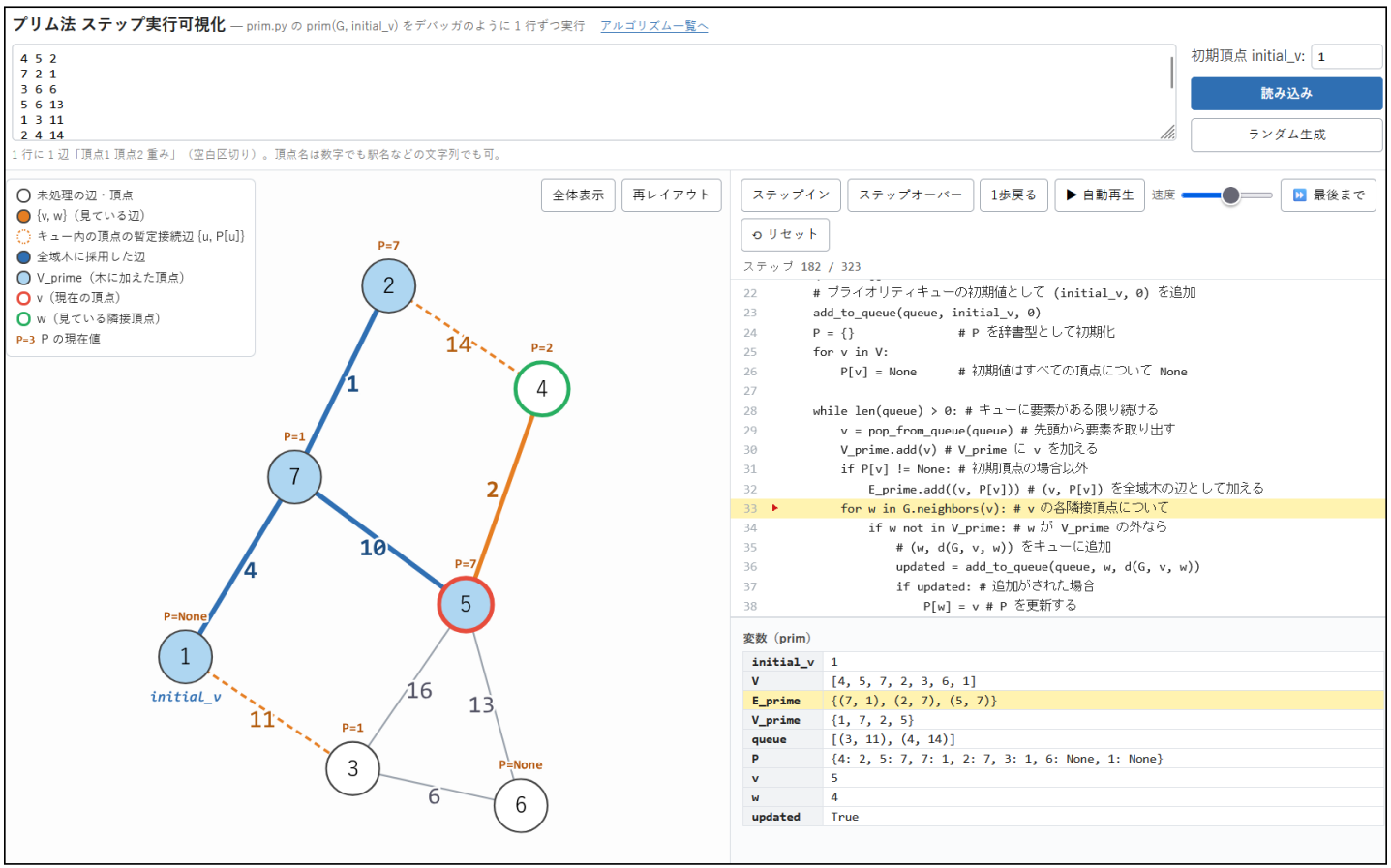
作例 3: 学生実験のハードウェア課題

計算機科学実験及演習 3（ハードウェア）

- FPGA 上での CPU 実装課題を解かせてみた
- Claude Opus 4.8 で一発で完成
  - 7 セグ表示以外は完璧に動作
- 70 ページのレポートも執筆
  - 合格レベル

作例 4: アルゴリズム可視化ツール

- 講義「グラフ理論」のアルゴリズムを可視化
- 手元で動かせる
  - 受講生が視覚的に学べる



作例 5: グラフ描画ツール

TikZ 形式で編集できるグラフ描画ツール

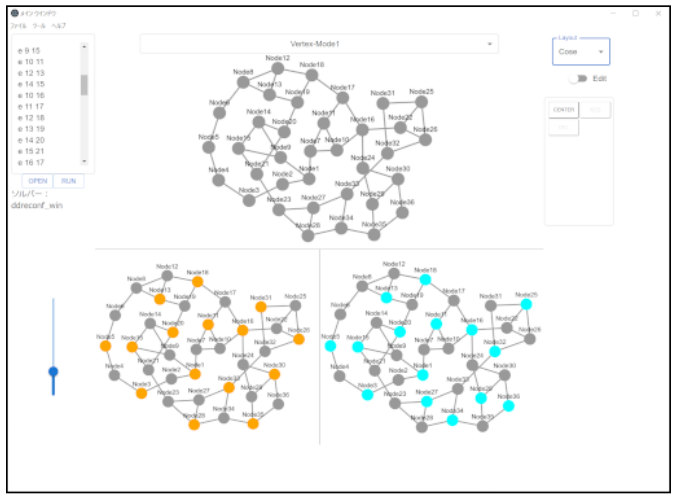
`https://github.com/junkawahara/graph_svg`

- Windows / Mac / Linux / Web ブラウザで動作
- 自分では使わなくなった
  - 論文の図は生成 AI に作らせるように
  - 動かしてみるとバグだらけ
- ツールを自作できるのがよい
  - かゆいところに手が届く

作例 6: CoReViewer

https://github.com/junkawahara/coreviewer-web-test

- 「組合せ遷移」問題のソルバー＋可視化ツール
- 2021 年に業者に〇〇万円で発注
  - デスクトップ版を Web へ移植
  - 生成 AI は既存コードの理解も得意
- 今の Claude Code なら 3 日で実装可能
  - しかも自分の思い通りに



デスクトップ版（https://github.com/junkawahara/coreviewer より）

作例 7: 実装困難な理論アルゴリズム

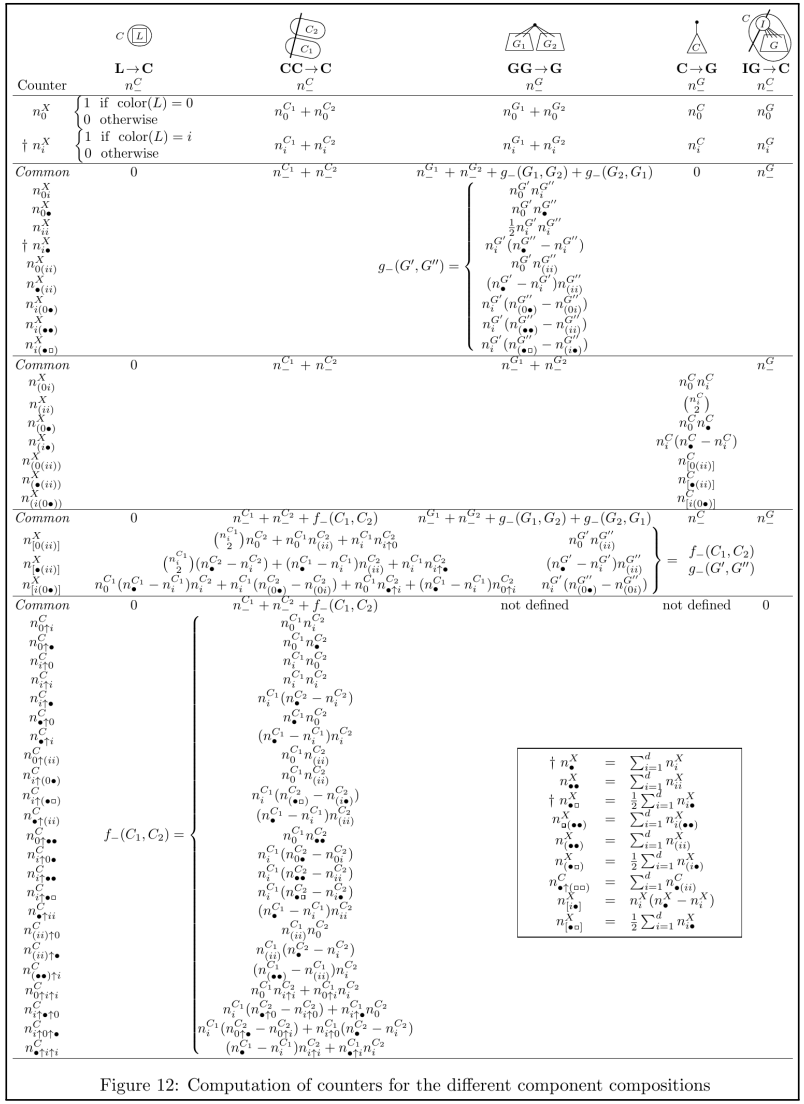
- 実装が困難なアルゴリズムもよくある
- 例: SODA 2013 の論文
  - 任意次数の 2 本の木の距離を高速計算

    トリプレット距離（根付き木・3 葉ずつ）

    カルテット距離（無根木・4 葉ずつ）
- Opus 4.8 でさえ実装できなかった
- 実装時間
  - Fable 5.1: 1 時間 40 分
  - Astra 6: 3 時間
  - 相互チェックでどちらも正しいと判定

G. S. Brodal, R. Fagerberg, T. Mailund, C. N. S. Pedersen, and A. Sand. Efficient algorithms for computing the triplet and quartet distance between trees of arbitrary degree. In *Proc. SODA 2013*, pp. 1814–1832, 2013.

作例 7: 論文中のアルゴリズムの図



Counter	$c\,(L)$ **L→C** n_-^C	C_2 C_1 **CC→C** n_-^C	G_1 G_2 **GG→G** n_-^G	C **C→G** n_-^G	C (L) G **IG→C** n_-^C
n_0^X	$\begin{cases}1 & \text{if } \mathrm{color}(L)=0\\ 0 & \text{otherwise}\end{cases}$	$n_0^{C_1}+n_0^{C_2}$	$n_0^{G_1}+n_0^{G_2}$	n_0^C	n_0^G
† n_i^X	$\begin{cases}1 & \text{if } \mathrm{color}(L)=i\\ 0 & \text{otherwise}\end{cases}$	$n_i^{C_1}+n_i^{C_2}$	$n_i^{G_1}+n_i^{G_2}$	n_i^C	n_i^G
Common	0	$n_-^{C_1}+n_-^{C_2}$	$n_-^{G_1}+n_-^{G_2}+g_-(G_1,G_2)+g_-(G_2,G_1)$	0	n_-^G
n_{0i}^X			$n_0^{G'}n_i^{G''}$		
$n_{0\bullet}^X$			$n_0^{G'}n_\bullet^{G''}$		
n_{ii}^X			$\frac{1}{2}n_i^{G'}n_i^{G''}$		
† $n_{i\bullet}^X$			$n_i^{G'}(n_\bullet^{G''}-n_i^{G''})$		
$n_{0(ii)}^X$			$n_0^{G'}n_{(ii)}^{G''}$		
$n_{\bullet(ii)}^X$			$(n_\bullet^{G'}-n_i^{G'})n_{(ii)}^{G''}$		
$n_{i(0\bullet)}^X$			$n_i^{G'}(n_{(i\bullet)}^{G''}-n_{(0i)}^{G''})$		
$n_{i(\bullet\bullet)}^X$			$n_i^{G'}(n_{(\bullet\bullet)}^{G''}-n_{(ii)}^{G''})$		
$n_{i(\bullet\square)}^X$			$n_i^{G'}(n_{(\bullet\square)}^{G''}-n_{(i\bullet)}^{G''})$		
Common	0	$n_-^{C_1}+n_-^{C_2}$	$n_-^{G_1}+n_-^{G_2}$		n_-^G
$n_{(0i)}^X$				$n_0^C n_i^C$	
$n_{(ii)}^X$				$\binom{n_i^C}{2}$	
$n_{(0\bullet)}^X$				$n_0^C n_\bullet^C$	
$n_{(i\bullet)}^X$				$n_i^C(n_\bullet^C-n_i^C)$	
$n_{(0(ii))}^X$				$n_{[0(ii)]}^C$	
$n_{(\bullet(ii))}^X$				$n_{[\bullet(ii)]}^C$	
$n_{(i(0\bullet))}^X$				$n_{[i(0\bullet)]}^C$	
Common	0	$n_-^{C_1}+n_-^{C_2}+f_-(C_1,C_2)$	$n_-^{G_1}+n_-^{G_2}+g_-(G_1,G_2)+g_-(G_2,G_1)$	n_-^C	n_-^G
$n_{[0(ii)]}^X$	$\binom{n_0^{C_1}}{2}n_0^{C_2}+n_0^{C_1}n_{(ii)}^{C_2}+n_i^{C_1}n_{i\dagger 0}^{C_2}$		$n_0^{G'}n_{(ii)}^{G''}$	$=$	$f_-(C_1,C_2)$
$n_{[\bullet(ii)]}^X$	$\binom{n_\bullet^{C_1}}{2}(n_\bullet^{C_2}-n_i^{C_2})+(n_\bullet^{C_1}-n_i^{C_1})n_{(ii)}^{C_2}+n_i^{C_1}n_{i\dagger\bullet}^{C_2}$		$(n_\bullet^{G'}-n_i^{G'})n_{(ii)}^{G''}$	$=$	$g_-(G',G'')$
$n_{[i(0\bullet)]}^X$	$n_0^{C_1}(n_\bullet^{C_1}-n_i^{C_1})n_i^{C_2}+n_i^{C_1}(n_{(0\bullet)}^{C_2}-n_{(0i)}^{C_2})+n_0^{C_1}n_{\bullet\dagger i}^{C_2}+(n_\bullet^{C_1}-n_i^{C_1})n_{0\dagger i}^{C_2}$		$n_i^{G'}(n_{(0\bullet)}^{G''}-n_{(0i)}^{G''})$		
Common	0	$n_-^{C_1}+n_-^{C_2}+f_-(C_1,C_2)$	not defined	not defined	0
$n_{0\dagger i}^C$	$n_0^{C_1}n_i^{C_2}$				
$n_{0\dagger\bullet}^C$	$n_0^{C_1}n_\bullet^{C_2}$				
$n_{i\dagger 0}^C$	$n_i^{C_1}n_0^{C_2}$				
$n_{i\dagger i}^C$	$n_i^{C_1}n_i^{C_2}$				
$n_{i\dagger\bullet}^C$	$n_i^{C_1}(n_\bullet^{C_2}-n_i^{C_2})$				
$n_{\bullet\dagger 0}^C$	$n_\bullet^{C_1}n_0^{C_2}$				
$n_{\bullet\dagger i}^C$	$(n_\bullet^{C_1}-n_i^{C_1})n_i^{C_2}$				
$n_{0\dagger(ii)}^C$	$n_0^{C_1}n_{(ii)}^{C_2}$				
$n_{i\dagger(0\bullet)}^C$	$n_0^{C_1}n_{(ii)}^{C_2}$				
$n_{i\dagger(\bullet\square)}^C$	$n_i^{C_1}(n_{(\bullet\square)}^{C_2}-n_{(i\bullet)}^{C_2})$				
$n_{\bullet\dagger(ii)}^C$	$(n_\bullet^{C_1}-n_i^{C_1})n_{(ii)}^{C_2}$				
$n_{0\dagger\bullet\bullet}^C$	$n_0^{C_1}n_{\bullet\bullet}^{C_2}$				
$n_{i\dagger 0\bullet}^C$	$n_i^{C_1}(n_{\bullet\bullet}^{C_2}-n_{0i}^{C_2})$				
$n_{i\dagger\bullet\bullet}^C$	$n_i^{C_1}(n_{\bullet\bullet}^{C_2}-n_{ii}^{C_2})$				
$n_{i\dagger\bullet\square}^C$	$n_i^{C_1}(n_{\bullet\square}^{C_2}-n_{i\bullet}^{C_2})$				
$n_{\bullet\dagger ii}^C$	$(n_\bullet^{C_1}-n_i^{C_1})n_{ii}^{C_2}$				
$n_{(ii)\dagger 0}^C$	$n_{(ii)}^{C_1}n_0^{C_2}$				
$n_{(ii)\dagger\bullet}^C$	$n_{(ii)}^{C_1}(n_\bullet^{C_2}-n_i^{C_2})$				
$n_{(\bullet\bullet)\dagger i}^C$	$(n_{(\bullet\bullet)}^{C_1}-n_{(ii)}^{C_1})n_i^{C_2}$				
$n_{0\dagger i\dagger i}^C$	$n_0^{C_1}n_{i\dagger i}^{C_2}+n_{0\dagger i}^{C_1}n_i^{C_2}$				
$n_{i\dagger 0\dagger 0}^C$	$n_i^{C_1}(n_{\bullet\dagger 0}^{C_2}-n_{i\dagger 0}^{C_2})+n_{ii\bullet}^{C_1}n_0^{C_2}$				
$n_{i\dagger 0\dagger\bullet}^C$	$n_i^{C_1}(n_{0\dagger\bullet}^{C_2}-n_{0\dagger i}^{C_2})+n_{i0\bullet}^{C_1}(n_{\bullet\bullet}^{C_2}-n_{i\bullet}^{C_2})$				
$n_{\bullet\dagger i\dagger i}^C$	$(n_\bullet^{C_1}-n_i^{C_1})n_{i\dagger i}^{C_2}+n_{\bullet\dagger i}^{C_1}n_i^{C_2}$				

$$
\begin{aligned}
\dagger\, n_\bullet^X &= \textstyle\sum_{i=1}^d n_i^X\\
n_{\bullet\bullet}^X &= \textstyle\sum_{i=1}^d n_{ii}^X\\
\dagger\, n_{\bullet\square}^X &= \tfrac{1}{2}\textstyle\sum_{i=1}^d n_{i\bullet}^X\\
n_{\square(\bullet\bullet)}^X &= \textstyle\sum_{i=1}^d n_{i(\bullet\bullet)}^X\\
n_{(\bullet\bullet)}^X &= \textstyle\sum_{i=1}^d n_{(ii)}^X\\
n_{(\bullet\square)}^X &= \tfrac{1}{2}\textstyle\sum_{i=1}^d n_{(i\bullet)}^X\\
n_{\bullet\dagger(\square\square)}^C &= \textstyle\sum_{i=1}^d n_{\bullet(ii)}^C\\
n_{[i\bullet]}^X &= n_i^X(n_\bullet^X-n_i^X)\\
n_{\bullet\square]}^X &= \tfrac{1}{2}\textstyle\sum_{i=1}^d n_{i\bullet}^X
\end{aligned}
$$

Figure 12: Computation of counters for the different component compositions

場合分けと数式がページ一面に（論文の Figure 12）

作例 7: 実装完成時の Fable の出力

SODA 2013 のアルゴリズム実装が完成しました。空のリポジトリから C++17 で論文の全アルゴリズムを実装し、オラクルとの一致を確認してコミット済みです。

実装内容（論文との対応）

- Section 3: 単一木の resolved/unresolved 三つ組・四つ組数（single_tree.cpp）。
- Section 4: HDT 構築。4 種の成分（L/I/C/G）と 5 アクション、ラウンド処理 i)〜iii)（hdt.cpp）。
- Section 5: Count(v) 再帰、contract（Steiner 木圧縮）と extract（HDT 根経路の和集合から側部分木をゼロ葉に畳む）（count.hpp）。再帰は明示スタックで深い木にも対応。
- Section 6/7: Figure 11・14 の全カウンタ（色ごとに疎に保持、2 色カウンタはペア表）、Figure 12・15 の合成規則、Figure 10・13・16 の寄与和を 9 ケースすべて実装（counters.hpp, contributions.hpp）。
- 公開 API（distance.hpp）で triplet/quartet 距離とパラメータ付き距離、CLI（build/soda13）も用意しました。

検証

- ナイーブ $O(n^3)/O(n^4)$ オラクルとの A〜E 全セル一致: triplet は $n \le 40$ で 2000 ケース、quartet は $n \le 16$ で 1200 ケース、高次数木でも一致。ASan/UBSan もクリーン。
- 各 HDT ノードの全カウンタを定義どおり総当たりで数えて照合するオラクル（tests/test_counters.cpp）も全一致。
- 計算量: time/($n \log n$) が triplet で n=1000〜256,000、quartet で $n \le$ 128,000 までほぼ一定。$\sqrt{n}$ 次数の星型でも d に比例した増加は見られません。

作例 8: グラフクラス認識・列挙ライブラリ

https://github.com/junkawahara/ai-coded-graph-recognition-test

- 75 種類超のグラフクラスに対応
  - 認識器 81 個（証明書付き）
  - 列挙器 115 個
- コードはすべて AI が生成
  - Claude Opus 4.6 ＋ Codex 5.2
  - 人間はタスク指示とレビューのみ
- C++11 ヘッダオンリー＋ Python
  - 約 8.6 万行（うちテスト約 2.5 万行）

作例 8: 対応しているグラフクラス

弦グラフ系	Chordal, Strongly chordal, Proper chordal, Split, Threshold, Weakly chordal, Block, Ptolemaic, Trivially perfect, Quasi-threshold, k-tree, Partial k-tree, k-degenerate
区間・円弧	Interval, Proper interval, Unit interval, Co-interval, Circular-arc, Proper circular-arc
順列・比較可能	Permutation, Comparability, Co-comparability, Bipartite permutation, Trapezoid
二部グラフ系	Bipartite, Chordal bipartite, Chain, Co-chain, Convex bipartite, Biconvex bipartite
平面グラフ系	Planar, Outerplanar, Maximal outerplanar, Cactus, Series-parallel, Apex, Maximal planar, Apollonian network, Cubic planar, Polyhedral, Simple quadrangulation, Halin, Fullerene
完全・構造的	Perfect, Cograph, Distance-hereditary, AT-free, Co-chordal, Line graph, Circle, Meyniel, Parity, Even-hole-free, Odd-hole-free, Cluster, Self-complementary
禁止誘導部分	Claw-free, Diamond-free, Triangle-free, Bull-free, P_5-free, Gem-free
葉べき	3-leaf power, 4-leaf power, 5-leaf power
木・森	Tree, Forest, Caterpillar, Unicyclic
連結性・正則性	Biconnected, Triconnected, Eulerian, k-regular, Cubic, Strongly regular, Snark, Laman
有向グラフ	Tournament, Directed graph, Poset

計 79 項目（README の表より）

「生成 AI のコーディング技法」？

- 本講演のタイトル:「生成 AI のコーディング技法」
- 伝えたいこと: そんな技法はほとんどない
  - 使い方は生成 AI に聞けばよい
- 生成 AI の tips はすぐに陳腐化
  - 例: GPT-3.5 時代に有効だった語句
    - 「あなたは〇〇のエキスパートです。」を先頭に
  - 今は無意味

生成 AI の性能はモデル次第

- 性能は条件によって大きく異なる
  - モデル・バージョン・時期・有料/無料
  - 「賢い/賢くない」はこれら次第
  - 他人の話はどのモデルかに注意
- 2026 年 10 月 9 日現在（私見）
  - コーディング最強: Claude Fable 5.1
  - 数学最強: ChatGPT Astra 6

    （どちらも月 100 ドル以上のプランのみ）
- 中国発オープンウェイトモデルも急進化
  - DeepSeek, Kimi, Qwen など

使用上の注意 1: ハルシネーション

- 生成 AI の出力は今でも誤っている可能性
- 例: 論文執筆時の引用作成

1. Deep Research で文献調査
2. 概要は人間が読み、ストーリーは人間が考える
3. ストーリーをもとに AI が執筆
4. .bib ファイルを AI が作成・引用
5. 文献の実在を出版社ページで確認させる
   (できれば複数の異なる AI で)
6. 最後は当然人間がチェック

「.bib に書かれた各文献が、出版社のページに存在するか調べて、情報が間違っていないかをチェックしてください。」

ハルシネーションの例

AIで調べた沖縄戦の判例、存在せず
掲載本を回収・絶版に

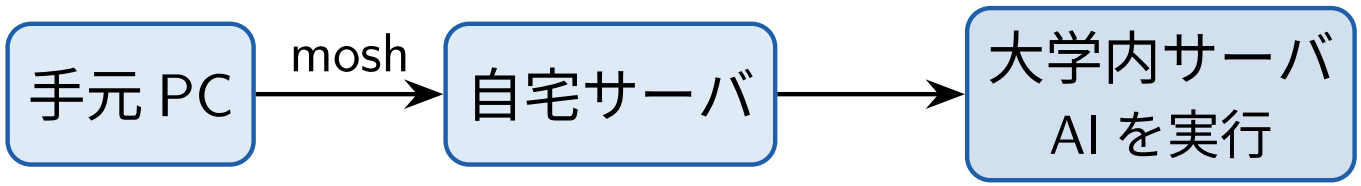
琉球大・島袋純教授、高文研から刊行

- 公開日: 2026年10月9日

https://www.okinawatimes.co.jp/articles/-/1941134

使用上の注意 2: PC の操作権限

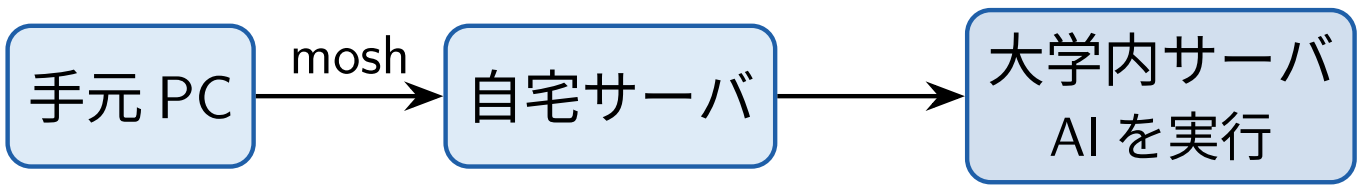
- AI に自分の PC の操作権限を与えている
- 発表者は重要ファイルのない別 PC で実行
  - 手元とほとんど変わらない操作感



- 先述の出席管理システムでも
  - 個人情報・パスワードはサーバごと隔離
  - 安全性の確保にはプログラミング経験が必要

使用上の注意 2: PC の操作権限

- AI に自分の PC の操作権限を与えている
- 発表者は重要ファイルのない別 PC で実行
  - 手元とほとんど変わらない操作感



- 先述の出席管理システムでも
  - 個人情報・パスワードはサーバごと隔離
  - 安全性の確保にはプログラミング経験が必要

まとめ

- 「技法」なんてものは存在しない
  - とはいえ、tips もいくつか紹介した
- 分からないことは目の前の生成 AI に聞けばよい
- 自分用のソフトウェアを作ってみよう